How The Apps Work Together:

A) Simplified to two main apps, logical grouping of related features
B) Users app handles authentication, profiles and user related functionalities
C) Projects app handles everything video-related

1."users" App (User Management)

```
users/
├── models.py        # User Profile model
├── views.py         # Login/Register logic
└── templates/       # User-related pages
```

Purpose-

1) Handles user accounts (signup/login)
2) Manages user profiles
3) Stores user information
4) Manages the 'user to user following' feature '
5) Handles authentication

Key Models- ***User , Profile , Follow***

2."projects" App (Core Features)

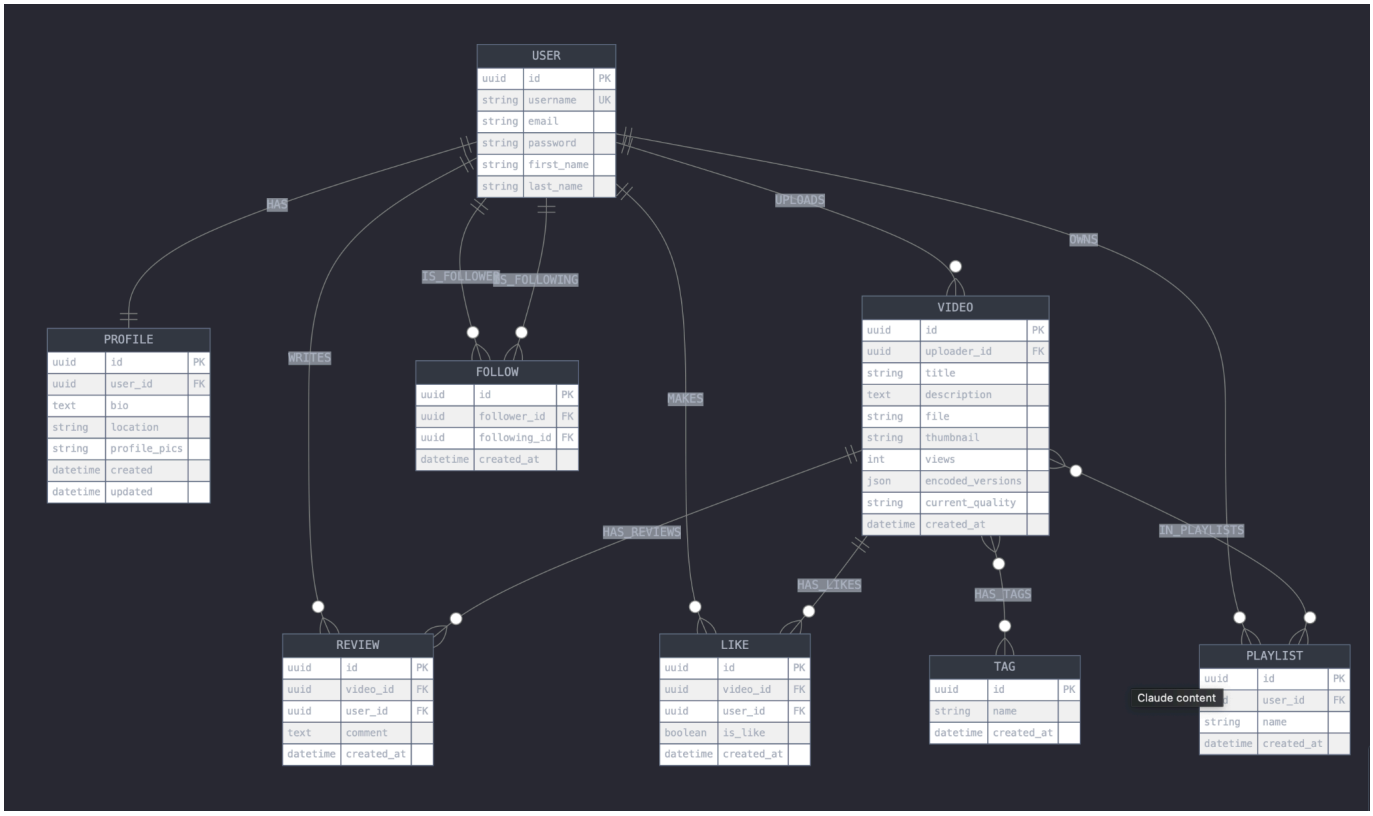
```
projects/
├── models.py        # Video storage/metadata
├── views.py         # Video upload/playback logic
└── templates/       # Video-related pages
```

Purpose-

1) Handles video uploads
2) Manages video playback
3) Stores video metadata
4) Manages video listings
5) Handles video reviews/ratings
6) Manages video tagging system
7) Thumbnail Processing

Key Models- ***Video, Tag, Like, Review, Playlist***

DB Schema



Key Relationships Between Tables:

1) User - Profile Relationship	2) User - Video Relationship	3) User - Review Relationship
Practical: Each user has exactly one profile, and each profile belongs to exactly one user Technical: One-to-One relationship with Profile.user_id referencing User.id Example: Like a YouTube account (User) having one channel profile (Profile) Access: user.profile or profile.user	Practical: One user can upload many videos, but each video has one uploader Technical: One-to-Many relationship with Video.uploader_id referencing User.id Example: A YouTuber uploading multiple videos to their channel Access: user.videos.all() or video.uploader	Practical: One user can write many reviews, each review has one author Technical: One-to-Many relationship with Review.user_id referencing User.id Example: A user commenting on multiple different videos Access: user.reviews.all() or review.user

4) User - Like Relationship Practical: One user can like many videos, each like comes from one user Technical: One-to-Many with Like.user_id referencing User.id Example: A user liking multiple different videos Access: user.video_likes.all() or like.user	5) User - Playlist Relationship Practical: One user can create many playlists, each playlist has one owner Technical: One-to-Many with Playlist.user_id referencing User.id Example: A user creating multiple playlists for different video categories Access: user.playlists.all() or playlist.user	6) Video - Tag Relationship Practical: One video can have many tags, one tag can be on many videos Technical: Many-to-Many relationship using an automatic junction table Example: A video having multiple tags (#funny, #music, #tutorial) Access: video.tags.all() or tag.video_set.all()
7) Video - Playlist Relationship Practical: One video can be in many playlists, one playlist can have many videos Technical: Many-to-Many relationship using an automatic junction table Example: Adding a music video to multiple different playlists Access: playlist.videos.all() or video.playlists.all()	8) User - Follow Relationship Practical: Users can follow many users and be followed by many users Technical: Many-to-Many through explicit Follow model with additional data Example: Like following multiple YouTube channels Access: user.following_set.all() or user.followers_set.all()	9) Video - Like Relationship Practical: One video can have many likes, each like belongs to one video Technical: One-to-Many with Like.video_id referencing Video.id Example: Multiple users liking or disliking a single video Access: video.likes.all() or like.video
10) Video - Review Relationship Practical: One video can have many comments, each comment belongs to one video Technical: One-to-Many with Review.video_id referencing Video.id Example: Like YouTube comments where multiple users can comment on a single video		

GITHUB WORKFLOW

1.Initial Status and Feature Branch Work # Check current branches (*indicates current branch) > git branch # Stage all changes > git add . # Commit changes with message > git commit -m "your message" # Push feature branch to remote > git push origin feature/branch-name	2. Update Main Branch # Switch to main branch > git checkout main # Pull changes from develop to main > git pull origin develop # Push changes to main > git push origin main #If rejected: # sync with remote main > git pull origin main # try pushing again > git push origin main	3.Update Develop Branch # Switch to develop > git checkout develop # Get latest changes > git pull origin develop # Merge feature branch into develop > git merge feature/branch-name # Push updated develop > git push origin develop
4.Clean Up Feature Branch # Delete local feature branch > git branch -d feature/branch-name # Delete remote feature branch > git push origin --delete feature/branch-name	5.Start New Feature Branch # Create and switch to new feature branch from develop > git checkout develop > git checkout -b feature/new-branch-name # Push new branch to remote and set tracking > git push -u origin feature/new-branch-name	6.Final Sync # Get all remote updates > git fetch --all # Pull updates for all branches > git pull --all # Verify branch structure > git branch